

Advanced classics

Vladislav Goncharenko

ML teamlead, Dzen



MIPT, spring 2024

Revise

Lecture 3: Image processing

- Image transforms
 - Affine
 - Homography
- Image gradients
- Morphology
 - Erosion
 - Dilation
 - Opening
 - Closing
 - etc...
- Skeletonization
- Canny edge detector
- Hough transform
- Watershed segmentation

Outline

1. Local keypoint detection
2. RANSAC
3. Background subtractors
4. Convolution implementation

Keypoints detection

girafe
ai

01

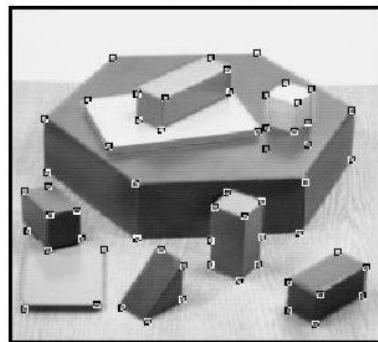
Keypoints

<https://disk.yandex.ru/i/Q5pcSN39G78KQA>

<https://www.cs.toronto.edu/~lindell/teaching/420/slides/lecture7.pdf>

Practice

https://github.com/ml-dafe/cv_mipt_minor/blob/seminar_05.ipynb



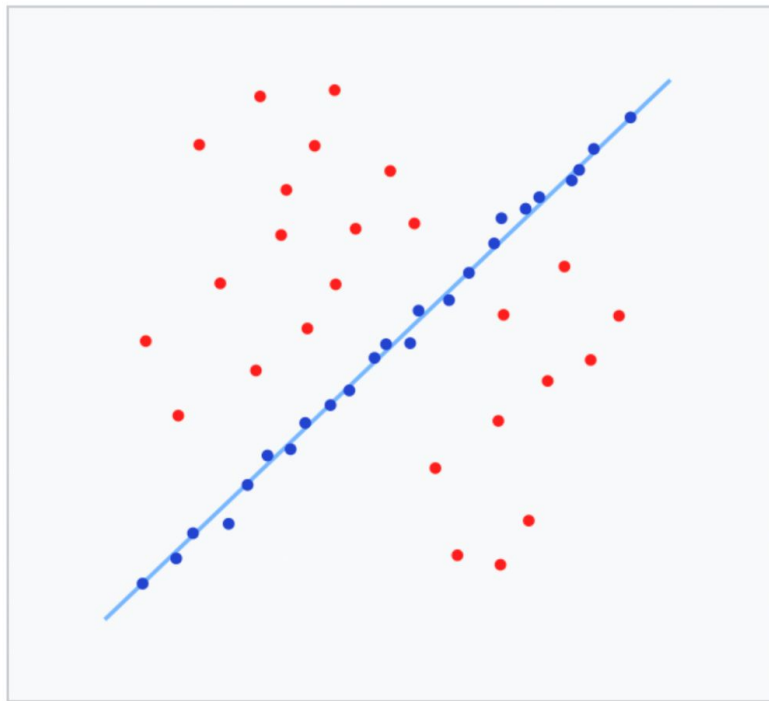
RANSAC

girafe
ai

02

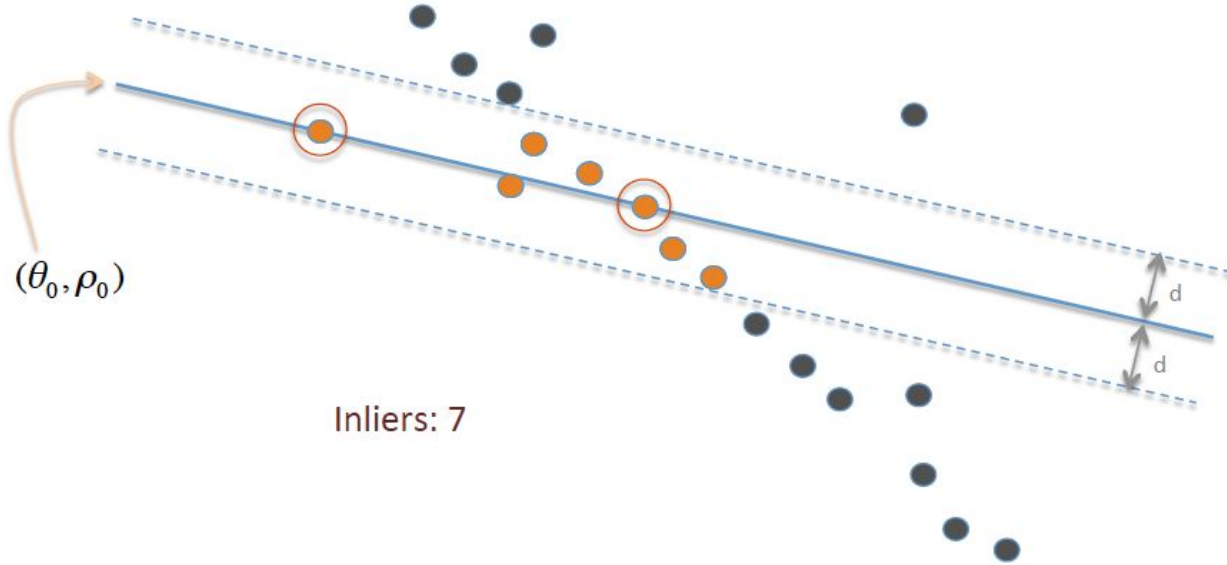
Random sample consensus

We need [robust algorithm](#) to detect some match (linear dependency)

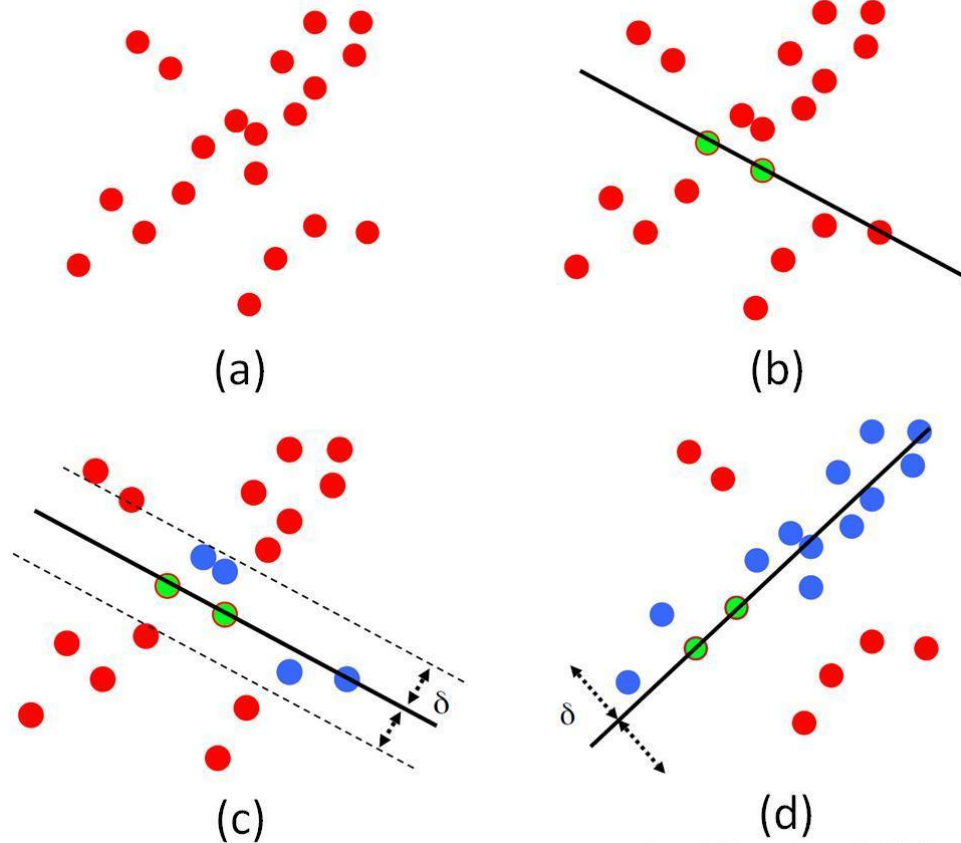


Random sample consensus

Try to fit dependency by a few points and then check if rest of data fits



Random sample consensus



Background subtractors

girafe
ai

03

Background subtractors

1. Problem statement
 - a. Problem itself
 - b. Adjacent tasks
2. Methods
 - a. Modelling stages
 - b. Naïve subtraction
 - c. MOG
 - d. MOG2
 - e. KNN
 - f. LSBP
 - g. CNT
 - h. Overview of DL methods
3. Comparison
 - a. Comparison papers
 - b. Manual inference comparison
4. Case studies/Usage

Problem statement



girafe
ai

Task overview (part 1)

Background subtraction (BS/BGS) - is a common and widely used technique for generating a foreground mask (namely, a binary image containing the pixels belonging to moving objects in the scene) by using static cameras.



Task overview (part 2)

Main presumptions:

- Static (or mostly static camera)
- Minimal training

One more example



Adjacent tasks

Main adjacent task - Moving Camera Background Modelling

Key difference:

- Static camera presumption is broken

Datasets: Davis 2017, SBI, SBM-RGBD etc. (more [here](#))

Noticeable paper: <https://arxiv.org/pdf/2209.07923v1.pdf>

Cool results to check: <https://github.com/bgu-cs-vil/deepmcbm>

Methods



Modelling stages

Overall in the production stages of background modelling are the following:

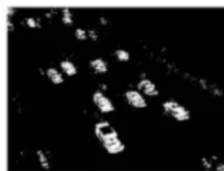
- Background initialization
- Background modelling
- Background maintenance
- Foreground detection

Naive subtraction (part 1)

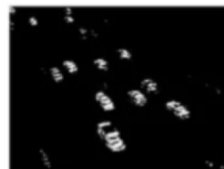
Naive subtraction is as naive as the following:

$$|Frame_i - Frame_{i-1}| > Threshold$$

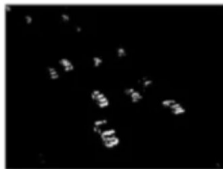
Th = 25



Th = 50



Th = 100



Th = 200



Naive subtraction (part 2)

Main problems:

- Too dependant on the threshold
- Reacts badly to lighting changes
- Reacts badly to shades
- Reacts badly to small camera movements

MOG

Paper: An Improved Adaptive Background Mixture Model for Realtime Tracking with Shadow Detection ([pdf](#))

Year: 2002

Authors: P. KaewTraKulPong and R. Bowden

OpenCV implementations: [CPU algorithm](#), [CUDA algorithm](#)

MOG (Algorithm outline, part 1)

1. Each pixel is modelled by a mixture of K gaussian distributions
2. Different Gaussians are assumed to represent different colours
3. The weight parameters of the mixture represent the time proportions that those colours stay in the scene
4. The background components are determined by assuming that the background contains B highest probable colours

MOG (Algorithm outline, part 2)

5. The probable background colours are the ones which stay longer and more static. Static single-colour objects trend to form tight clusters in the colour space while moving ones form widen clusters due to different reflecting surfaces during the movement
6. Every new pixel value is checked against existing model components in order of fitness (new)
7. New shadow detection heuristic (new)

MOG (Important math)

Each pixel in the scene is modelled by a mixture of K Gaussian distributions. The probability that a certain pixel has a value of $\mathbf{x}_N$ at time N can be written as

$$p(\mathbf{x}_N) = \sum_{j=1}^K w_j \eta(\mathbf{x}_N; \boldsymbol{\theta}_j)$$

w_k / σ_k fitness value

MOG (pictures)

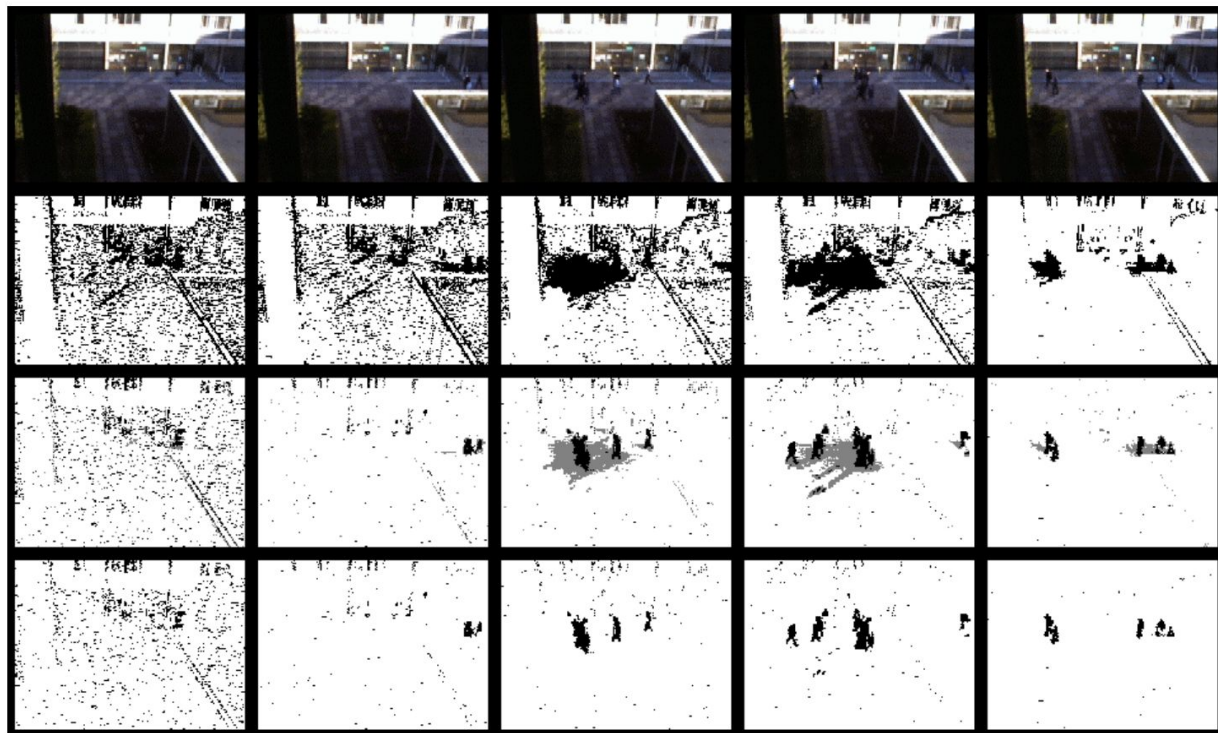


Figure 1: The top row displays the original sequence at frames 15, 105, 235, 290 and 1200 respectively. The second row shows the results from Grimson et al.'s. The last two rows are the results of our proposed method with and without moving shadows displayed in the images. The shadows are shown in grey.

MOG2

Papers:

- Efficient adaptive density estimation per image pixel for the task of background subtraction (2006, Zoran Zivkovic and Ferdinand van der Heijden, [link](#))
- Improved adaptive gaussian mixture model for background subtraction (2004. Zoran Zivkovic, [link](#))

OpenCV implementations: [link](#)

MOG2 (Algorithm outline)

There is 1 the most important feature - number K is chosen algorithmically

Math here is pretty hard (oh, these pre-NN times)

Basically before it was fixed number K , then it is chosen based on Gaussian probability density

MOG2 (Pictures)



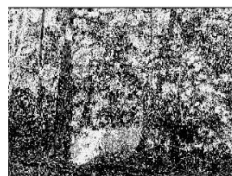
'traffic' sequence
average processing time per frame Old: 19.1ms New: 13.0ms



'lab' sequence
average processing time per frame Old: 19.3ms New: 15.9ms



'trees' sequence
average processing time per frame Old: 19.7ms New: 19.3ms



KNN

Paper: Efficient adaptive density estimation per image pixel for the task of background subtraction (2006, Zoran Zivkovic and Ferdinand van der Heijden, [link](#))

Official OpenCV implementation: [link](#)

It is usually very efficient in case foreground image size is very small.

KNN (algorithm outline)

In the paper it's called "balloon variable kernel density estimation"

$$\hat{p}_{\text{non-parametric}}(\vec{x}|\mathcal{X}_T, \mathbf{BG}) \approx \frac{1}{TV} \sum_{m=t-T}^t b^{(m)} \mathcal{K} \left(\frac{\|\vec{x}^{(m)} - \vec{x}\|}{D} \right).$$

K - kernel,

D is dynamically chosen set

CNT

Paper: There is no actual paper!

Algorithm was provided by Sagi Zeevi in his github repo (<https://github.com/sagi-z/BackgroundSubtractorCNT>)

But it is also implemented in OpenCV: [link](#)

Pros: **super** fast

CNT (algorithm outline)

It is basically a counter of frames any pixel is in the image.
That's pretty much it.

DL-based methods

Link to a great paper - <https://arxiv.org/pdf/2105.01342.pdf>

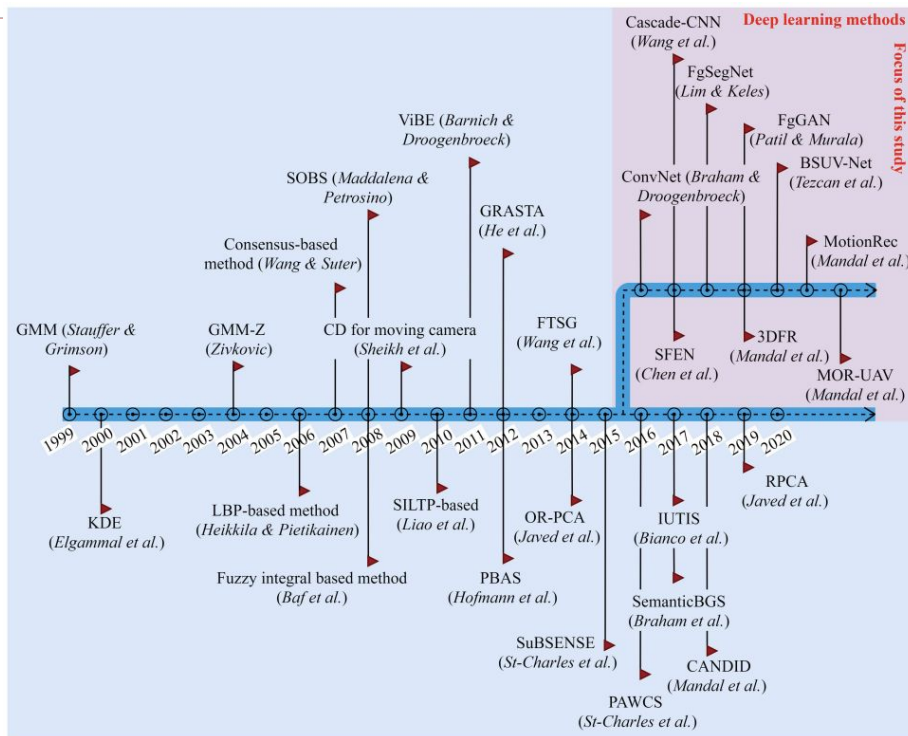
TL;DR - basically it is a lot of different versions of CNN

Main issue - need for labelled dataset

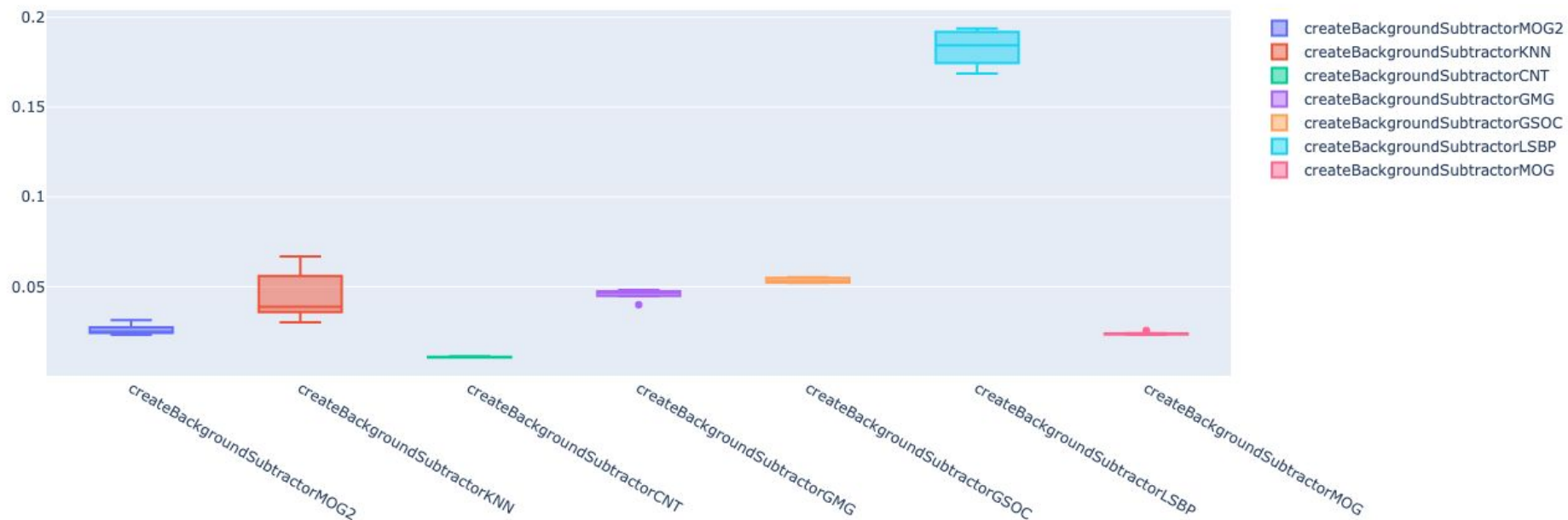
Comparison



History of methods



Inference comparison



Comparison of pictures

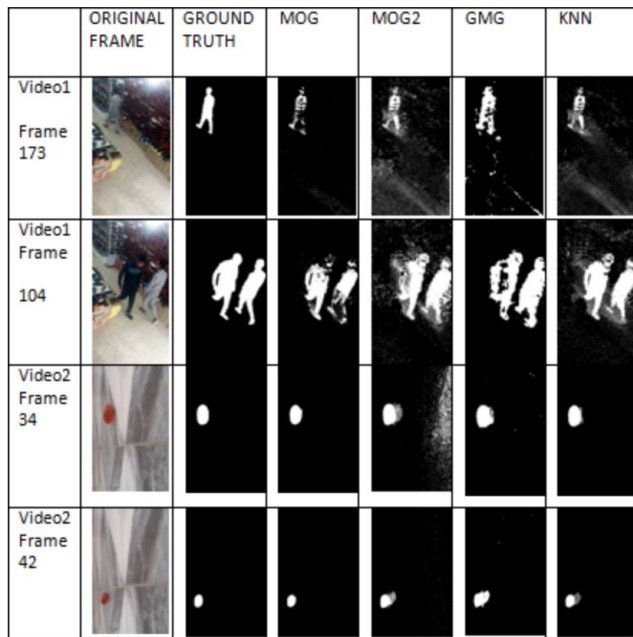


Fig. 1. Comparison the motions detected by different BS algorithms.

Case studies



Use cases overview

There is a great paper with an overview of different methods across all possible use cases and industries:

Background Subtraction in Real Applications: Challenges, Current Models and Future Directions (Thierry Bouwmans, Belmar Garcia-Garcia), [link](#)

Most interesting insights

1. Most of the tasks are solved with basic MOG (sometimes KNN), it may be because article are outdated
2. Honeybees are observed with KNN (cause they are small, just like we discussed)
3. Camera jitters is still a challenge for such methods, even with the current state of DL models

Efficient convolution implementation

girafe
ai

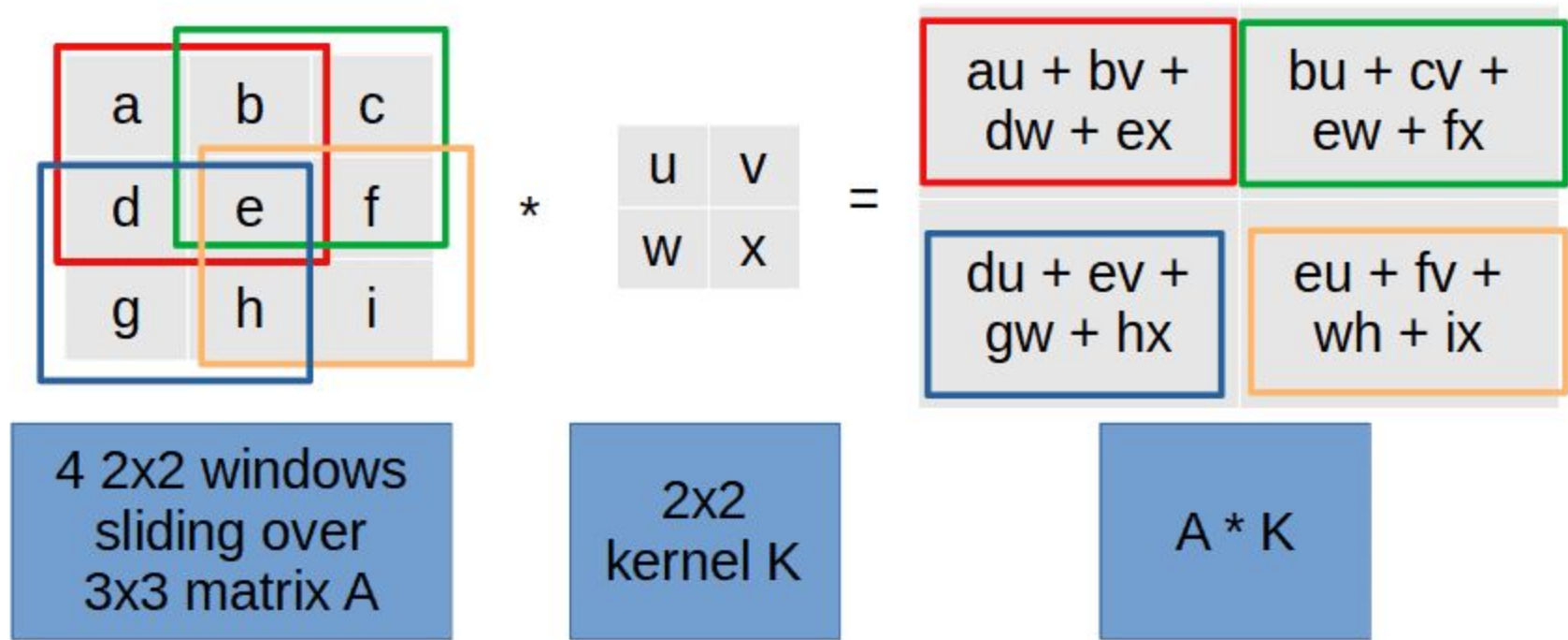
04

Profile of convolution operation

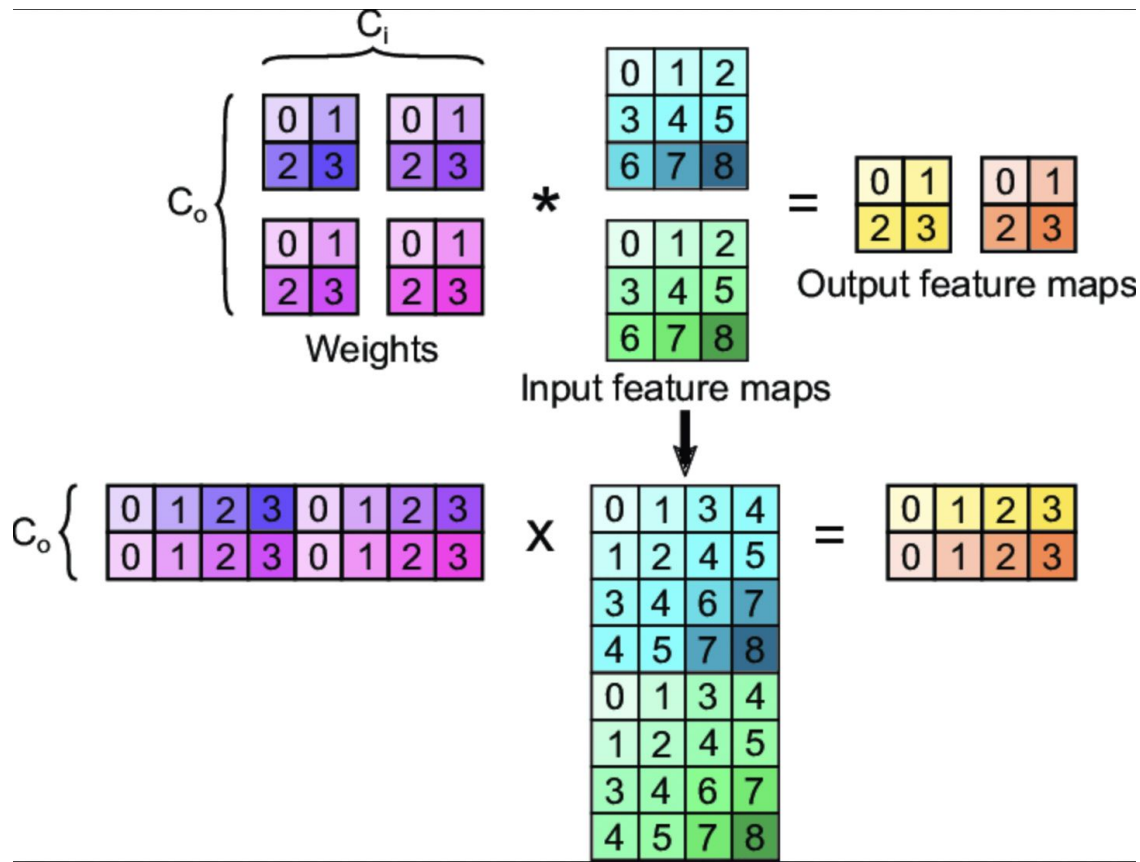


Name	Self CPU total %	Self CPU total	CPU total %	CPU total	CPU time avg
mkldnn_convolution	76.66%	6.381ms	76.66%	6.381ms	6.381ms
addmm	11.92%	992.389us	11.92%	992.389us	992.389us
relu	10.58%	880.450us	10.58%	880.450us	880.450us
_convolution	0.30%	24.680us	76.99%	6.409ms	6.409ms
unsigned short	0.16%	13.714us	0.16%	13.714us	13.714us
as_strided	0.16%	13.134us	0.16%	13.134us	13.134us
reshape	0.09%	7.699us	0.25%	20.833us	20.833us
conv2d	0.05%	4.510us	77.09%	6.417ms	6.417ms
convolution	0.04%	3.619us	77.03%	6.413ms	6.413ms
contiguous	0.03%	2.877us	0.03%	2.877us	0.959us
Self CPU time total: 8.324ms					

Convolution implementation



Convolution implementation



Convolution implementation



8	9	4	4
---	---	---	---

Flat Kernel



Original Kernel:

8	9
4	4

Original Input Matrix:

3	9	0
2	8	1
1	4	8

3	9	2	8
9	0	8	1
2	8	1	4
8	1	4	8



Window 1
Window 2
Window 3
Window 4

for col in range(input.shape[1] - 2):

Memory overhead

Size of image is smaller than space
used to store it on disk!

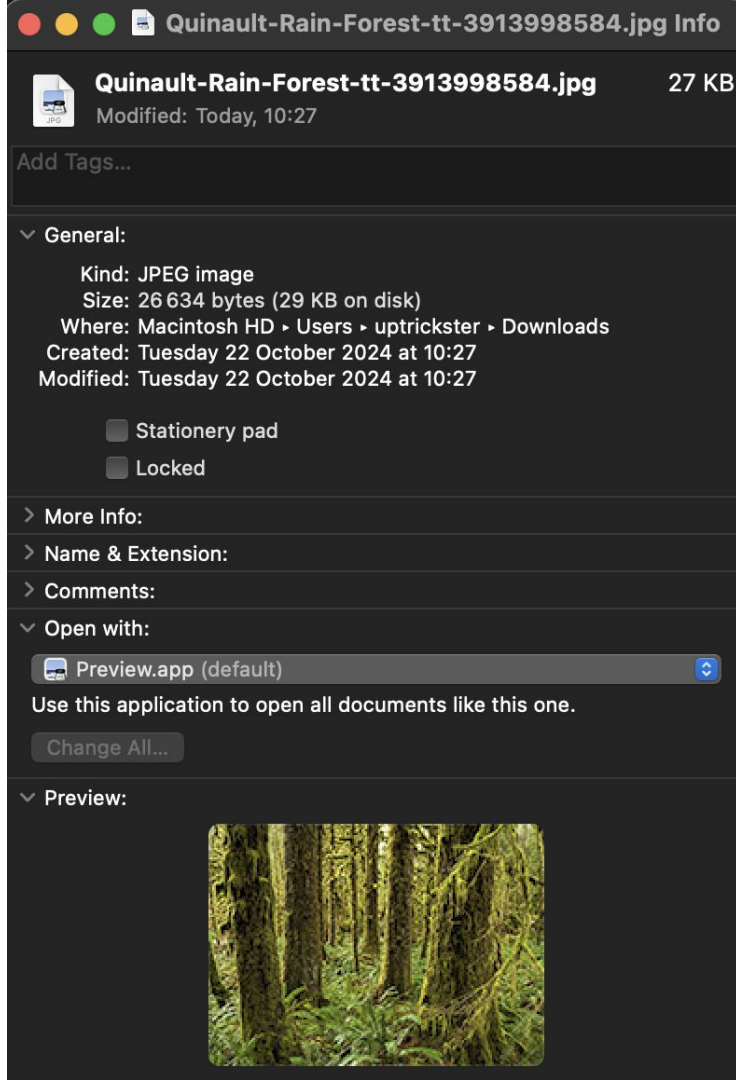
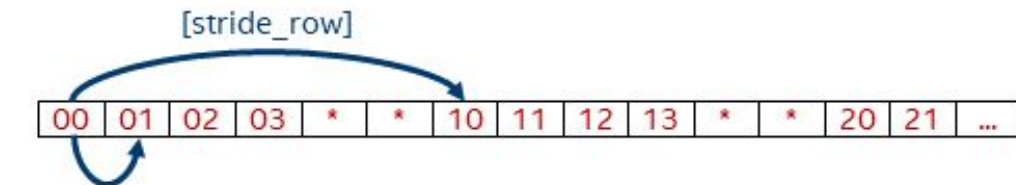
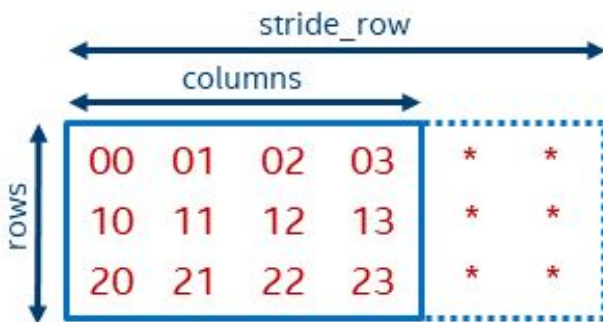


Image strides



Strides



[stride_col = 1]

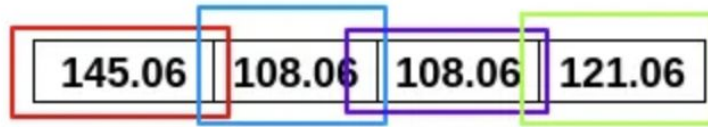
$$\begin{aligned}\text{map}(r, c) &= r * \text{stride_row} + c * \text{stride_col} \\ &= r * \text{stride_row} + c\end{aligned}$$

Convolution implementation



Result after Matrix Multiplication + bias:

Bias 0.06



Reshaping :



Convolution implementation

1. paywalled article:
<https://towardsdatascience.com/how-are-convolutions-actually-performed-under-the-hood-226523ce7fbf> ([cached version](#))
2. notebook with implementation
https://colab.research.google.com/drive/19lj1YXHyeRe9vocY_Z1bK2ZJ_OxcsBX#scrollTo=0TPIBcphTOA-
3. free article:
<https://medium.com/@sundarramanp2000/different-implementations-of-the-ubiquitous-convolution-6a9269dbe77f>
4. <https://stackoverflow.com/questions/53097952/how-to-understand-numpy-strides-for-layman> - memory strides explained
5. <https://pytorch.org/docs/stable/generated/torch.nn.Unfold.html>
<https://pytorch.org/docs/stable/generated/torch.nn.Fold.html>

Thanks for attention!

Questions?

